Supplementary Methods IV

Development and comparison of multilayer perceptron architectures for macro-environment prediction

Multilayer perceptrons (MLPs) were trained using the 1,920-dimensional mean component of the Evo 2 1B embeddings to investigate whether the sequence-derived representation could be used to predict the four macro-environments.

Model development was performed in successive stages using PyTorch for model implementation and Optuna for hyperparameter optimisation. Preliminary searches were first used to investigate relatively shallow networks by varying the number and width of the hidden layers together with dropout, batch normalisation and the main training parameters. These analyses were followed by a more focused search, from which a configuration containing three hidden layers with 512, 256 and 32 units was retained according to validation performance. A linear output layer returned the four macro-environment scores. This configuration is referred to as the 4-layer MLP in the main text and figures (the output layer was considered when the name was given).

The shallow architecture was then extended to investigate whether increasing network depth improved prediction. Two additional networks containing seven and twelve hidden layers were considered. Rather than repeating the complete search over both depth and width, the hidden dimensions of these networks were specified in advance and progressively reduced towards the output. The subsequent Optuna searches therefore focused on dropout, batch normalisation and training hyperparameters. The seven-hidden-layer network contained 1,024, 512, 256, 128, 64, 32 and 16 units, whereas the twelve-hidden-layer network contained 1,536, 1,536, 1,024, 1,024, 768, 768, 512, 512, 256, 256, 128 and 64 units.

Optuna trials were ranked according to validation cross-entropy loss. Validation accuracy, macro precision and macro F1 scores were also recorded and validation macro F1 was used to compare the retained network depths because it assigned equal importance to the four unequally represented macro-environments. The held-out test set was evaluated only after the architecture and training configuration had been fixed. The search spaces used in the final optimisation stages are reported in Table IV-SM1.

Table IV-SM1. **Hyperparameter spaces used for optimisation of the MLP architectures.** Continuous learning-rate and weight-decay parameters were sampled on a logarithmic scale. The number of trials refers to the Optuna study used for each retained architecture.

Optimisation stage	Hyperparameter	Search space
4-MLP optimisation	Number of hidden layers	1–3
	Candidate hidden dimensions	Decreasing combinations selected from 512, 256, 128, 64, 32 and 16
	Dropout rate	0.30–0.5
	Batch normalisation	True or false; set to false for batch sizes below 128
	Learning rate	9×10^{-5} - 10^{-3}
	Weight decay	10^{-5} - 10^{-4}
	Batch size	32, 64, 128 or 256
	Early-stopping patience	100–250 epochs
	Optuna trials	50
7-hidden-layer MLP optimisation	Hidden dimensions	Fixed at 1,024, 512, 256, 128, 64, 32 and 16
	Middle-layer dropout	0.10–0.25
	Late-layer dropout	0.02–0.10
	Batch normalisation	True or false
	Learning rate	10^{-6} - 5×10^{-4}
	Weight decay	10^{-5} - 10^{-2}
	Batch size	32, 64, 128 or 256
	Early-stopping patience	20–50 epochs

	Optuna trials	75
12-hidden-layer MLP optimisation	Hidden dimensions	Fixed at 1,536, 1,536, 1,024, 1,024, 768, 768, 512, 512, 256, 256, 128 and 64
	Middle-layer dropout	0.10–0.21
	Late-layer dropout	0.01–0.61
	Batch normalisation	True or false
	Learning rate	10^{-6} - $5\times\ 10^{-4}$
	Weight decay	10^{-5} - 10^{-2}
	Batch size	32, 64, 128 or 256
	Early-stopping patience	20–50 epochs
	Optuna trials	50

The three retained architectures were retrained using the fixed metagenome-aware training, validation and test splits. Each embedding dimension was standardised using the mean and standard deviation calculated exclusively from the training set. The fitted transformation was subsequently applied without modification to the validation and test sets. No L2 normalisation was applied to the individual embedding vectors. Each hidden transformation consisted of a linear layer followed by ReLU activation. Hidden-layer weights were initialised using Kaiming-uniform initialisation and their biases were set to zero. The output-layer bias was also initialised to zero. Batch normalisation was applied after all hidden linear transformations except the first, and dropout was applied according to the architecture-specific configuration reported in Table IV-SM2. The final linear layer returned four logits corresponding to Aquatic, Terrestrial, Engineered and Host-associated genomes. Softmax was applied only to convert these logits into class probabilities during evaluation. The models were trained using weighted multiclass cross-entropy loss, with class weights calculated from the training set using the balanced weighting procedure implemented in scikit-learn. The assigned weights were inversely proportional to the frequency of each macro-environment and were used in the multiclass cross-entropy loss for all three architectures.

For the final comparison on the fixed metagenome-aware splits, all three retained MLP architectures were retrained using the same optimisation settings. Training used AdamW for a maximum of 300 epochs with a batch size of 128, a learning rate of 1×10^{-4} and a weight decay of 7×10^{-5}. Training batches were shuffled at each epoch, whereas validation and test batches were kept in a fixed order. Early stopping monitored validation cross-entropy loss with a patience of 25 epochs and a minimum improvement of 1×10^{-3}. For each architecture, the parameter state associated with the lowest validation loss was retained and used for the final evaluation. All runs used a random seed of 42.

Table IV-SM2. **Architecture-specific configurations used for the final MLP analysis.** Hidden dimensions are reported in their order from the input representation to the four-unit output layer. Training parameters shared by the three models are reported in the text.

Model	Hidden dimensions	Batch normalisation	Dropout applied during final training
4-layer MLP	512, 256, 32	After the second and third hidden layers	0.49 after each hidden layer
7-hidden-layer MLP	1,024, 512, 256, 128, 64, 32, 16	After all hidden layers except the first	0.15 after hidden layers four to seven
12-hidden-layer MLP	1,536, 1,536, 1,024, 1,024, 768, 768, 512, 512, 256, 256, 128, 64	After all hidden layers except the first	0.10 after the first three hidden layers and 0.21 after the remaining hidden layers

The three models were evaluated using accuracy, macro precision, macro F1 and class-specific F1 scores. Row-normalised confusion matrices were additionally generated to examine the distribution of predictions within each true macro-environment.

Development and tuning of the multimodal two-tower models

Architecture and input-specific transformations

Evo 2 embeddings and KO copy-number profiles were processed by separate branches because the two inputs differed in dimensionality, numerical distribution and biological meaning. This parallel structure allowed each modality to be transformed using a separate set

of parameters before integration. Embedding dimensions were standardised using the means and standard deviations estimated from the training set, and the fitted transformation was applied unchanged to the validation and test sets. KO copy numbers were transformed using log1p.

Each tower contained two hidden transformation blocks followed by a final linear projection. The first block consisted of a 512-unit linear layer, LayerNorm, GELU activation and dropout. The second contained a 128-unit linear layer followed by LayerNorm and GELU activation. A final linear layer projected the representation to 64 dimensions. Dropout was set to 0.10 in the embedding tower and 0.20 in the KO tower and was applied after the first hidden block. GELU was used to provide a smooth nonlinear transformation while retaining small negative activations, rather than setting all negative values to zero as ReLU does. The 64-dimensional outputs of the two towers were independently normalised to unit L2 norm before fusion. This prevented differences in vector magnitude from determining the relative contribution of the two modalities. Their contribution was subsequently controlled by a trainable scalar parameter.

The scalar was transformed using a sigmoid function to obtain a modality weight, α, bounded between zero and one. The normalised embedding representation was multiplied by α and the normalised KO representation by $1 - \alpha$, after which the two weighted vectors were concatenated into a 128-dimensional joint representation. The raw scalar parameter was initialised to 0.5, corresponding to an initial effective embedding weight of approximately 0.62 after the sigmoid transformation. Its value was subsequently learned during training and was not treated as a tuned hyperparameter. The joint representation was provided to a linear classification layer in the three-class and direct four-class models, or to two linear classification heads in the multitask model.

Staged development of the integration strategy

Development of the multimodal model started from direct concatenation of the representations generated by the embedding and KO towers. Subsequent experiments evaluated different approaches to control the scale and relative contribution of the two modalities (*Table IV-SM3*). These included normalisation of the tower outputs before concatenation, normalisation of the KO representation, modality dropout, a learned gating mechanism and scalar-weighted concatenation. A delayed-update strategy was examined separately because it modified the training schedule rather than the fusion operation.

Table IV-SM3. **Integration and training strategies examined during development of the three-class two-tower model.**

Strategy	Implementation and purpose	Validation macro F1	Outcome
Direct concatenation	The two tower outputs were concatenated without explicit control of their scale or relative contribution.	0.807	Initial configuration
Normalisation before concatenation	Both tower outputs were normalised before concatenation to reduce differences in vector magnitude.	0.799	Evaluated during development
KO-output normalisation	The KO representation was normalised before concatenation to reduce its scale relative to the embedding representation.	0.853	Used as the basis for subsequent integration experiments
Learned gating	A learned gate controlled the contribution of the two tower representations.	0.831	Not retained
Modality dropout	One modality was randomly removed during training to reduce dependence on a single input.	0.849	Not retained
Learned scalar weighting	A trainable scalar controlled the relative contribution of the embedding and KO representations.	0.852	Retained
Delayed KO updating	The KO representation contributed to prediction but its tower was not updated during the first training epochs.	0.851	Retained and subsequently combined with scalar weighting

The different configurations were evaluated using validation macro F1. Since they were developed sequentially, some experiments also incorporated modifications retained from previous stages and should not be interpreted as a controlled ablation in which only one component changed. Nevertheless, they were used to identify which integration mechanisms

should be carried forward. The reported macro F1 corresponds to the validation performance of the checkpoint retained according to validation loss.

Normalisation of the KO representation produced a substantial improvement over the initial configurations, indicating that differences in the scale of the tower outputs affected their integration. Modality dropout and learned scalar weighting produced similar validation performance, whereas the gating mechanism did not provide an improvement. Scalar weighting was retained because it allowed the relative contribution of the two modalities to be learned using a single parameter and could be combined directly with normalisation of the tower outputs.

The delayed-update strategy was motivated by the different learning dynamics observed for the two modalities. KO-only models reached their minimum validation loss considerably earlier than embedding-only models. During the delayed phase, the KO representation was therefore detached from the computational graph: it continued to contribute to classification, but the parameters of the KO tower did not receive gradient updates. Both towers were subsequently optimised jointly. Delayed KO updating was first evaluated independently and was later combined with learned scalar weighting in the retained architecture.

In the final integration procedure, the 64-dimensional outputs of both towers were independently normalised to unit L2 norm. A trainable scalar was transformed using a sigmoid function to obtain a modality weight, α, between zero and one. The embedding representation was multiplied by α and the KO representation by $1 - \alpha$, after which the two weighted vectors were concatenated into a 128-dimensional joint representation. The raw scalar was initialised to 0.5, corresponding to an initial effective embedding weight of approximately 0.62 after the sigmoid transformation. The parameter was learned during training and was not included in the hyperparameter search.

Optimisation of the three-class model

The first structured optimisation was performed for the three-class problem in which Aquatic and Terrestrial genomes were combined into the Environmental class. An initial fixed-split search evaluated 120 configurations obtained by varying the projection dimensions, learning rate, label smoothing and number of delayed KO epochs reported in Table IV-SM4. A subsequent search examined a broader range of projection dimensions and learning rates across 576 configurations while retaining the learned scalar modality weight.

Model selection was based on validation performance. The configuration carried forward for the subsequent comparisons used 64-dimensional projections for both branches, a learning rate of 1×10^{-4}, weight decay of 1×10^{-3}, label smoothing of 0.01 and three delayed KO epochs.

Table IV-SM4. **Search spaces used for optimisation of the three-class two-tower model**.

Parameter	Fixed-split search	Expanded learned-weight search	Retained value
Embedding projection dimension	64, 128, 256	32, 64, 128, 256	64
KO projection dimension	32, 64	32, 64, 128, 256	64
Embedding dropout	0.10	0.10	0.10
KO dropout	0.20	0.20	0.20
Learning rate	3×10^{-5}, 1×10^{-4}	3×10^{-5}, 1×10^{-4}, 3×10^{-4}	1×10^{-4}
Weight decay	1×10^{-3}	1×10^{-3}	1×10^{-3}
Label smoothing	0.01, 0.05	0.01, 0.05	0.01
Delayed KO epochs	3, 5, 10, 15, 25	3, 5, 10, 15, 25, 50	3
Batch size	144	144	144
Early-stopping patience	15	15	15
Minimum validation-loss improvement	1×10^{-3}	1×10^{-3}	1×10^{-3}

Direct and multitask macro-environment formulations

After defining the multimodal representation, two formulations were evaluated for prediction of the four macro-environments. The direct model used a single linear classification head to predict Aquatic, Terrestrial, Engineered and Host-associated genomes. The multitask model used two linear heads connected to the same 128-dimensional fused representation. The upper head predicted Environmental, Engineered and Host-associated, whereas the lower head distinguished Aquatic from Terrestrial genomes within the Environmental group.

The lower-task target was defined only for Aquatic and Terrestrial genomes. For Engineered and Host-associated genomes, the target was assigned an ignored value and did not contribute to the lower-task loss. Class weights were calculated separately for the two tasks from the corresponding training-set frequencies using the balanced weighting procedure implemented in scikit-learn. Label smoothing of 0.01 was applied to the upper-task cross-entropy loss, whereas no label smoothing was applied to the Aquatic–Terrestrial loss.

The total training objective combined the upper- and lower-task cross-entropy losses. Equal weighting and configurations giving greater relative importance to either task were compared using validation macro F1 (*Table IV-SM5*). Increasing the contribution of the upper task improved its validation macro F1 but reduced performance on the Aquatic–Terrestrial task. Increasing the relative contribution of the lower task did not improve its validation performance. Equal weighting was therefore retained because it produced the highest validation macro F1 for the Aquatic–Terrestrial distinction while maintaining good performance on the upper task.

Experiments also evaluated separate modality-weight parameters for the two classification heads. Since they did not provide a consistent validation improvement across both tasks, the simpler shared modality weight was retained. These experiments were considered during model development but were not treated as a separate modelling strategy in the Results.

Table IV-SM5. **Comparison of the loss-weighting configurations evaluated for multitask training.**

Configuration	Upper-task loss weight	Lower-task loss weight	Upper-task validation macro F1	Aquatic–Terrestrial validation macro F1	Outcome
Equal	1.0	1.0	0.784	0.756	Retained
MoreMacro	1.0	0.5	0.803	0.745	Not retained
MoreEnv	0.5	1.0	0.783	0.749	Not retained

Final training and evaluation

The final multitask model was trained for a maximum of 300 epochs using AdamW with a learning rate of 1×10^{-4}, weight decay of 1×10^{-3} and a batch size of 144. The learning rate was reduced by a factor of 0.5 after three epochs without improvement in total validation loss, to a minimum of 1×10^{-6}. Early stopping monitored total validation loss with a patience of 15 epochs and a minimum improvement of 1×10^{-3}. The parameter state associated with the lowest total validation loss was retained.

Alternative configurations and the direct and multitask formulations were compared using validation metrics. The test set was not used as a criterion for model selection. Accuracy, macro precision and macro F1 were subsequently calculated on the test set. For the multitask model, metrics were reported separately for the upper and lower tasks because they addressed different classification levels and did not directly produce a single four-class output. Class-specific true-positive rates were calculated from row-normalised confusion matrices and corresponded to recall for each class.

Hyperparameter optimisation for sub-environment models

Associations with the 26 sub-environments were formulated as a multilabel problem and modelled through 26 independent one-vs-rest binary classifiers. Logistic regression (LR) was implemented with scikit-learn SGDClassifier using log loss and elastic-net regularisation. Random Forest (RF) models were implemented with scikit-learn RandomForestClassifier. Hyperparameters were selected independently within each annotation-system and model-family analysis using exhaustive grid search and three-fold grouped multilabel-stratified cross-validation. MAGs originating from the same metagenome were kept in the same fold. For stratification, labels were combined at metagenome level and the group-level multilabel matrix was supplied to MultilabelStratifiedKFold.

Following hyperparameter selection, one binary classifier per sub-environment was refitted using the complete training set. The class-specific decision thresholds were determined from the out-of-fold probabilities generated during cross-validation and subsequently applied unchanged to the held-out test set. Five sets of predictions were retained for comparison: in-fold predictions for the samples used to train each cross-validation model (Train CV), out-of-fold predictions for the corresponding validation folds (Validation CV), held-out test-set predictions averaged across the cross-validation models (Test CV), predictions for the complete training set obtained from the refitted models (Train Refit) and predictions of the same refitted models on the held-out test set (Test Refit). Train Refit predictions were used only to examine model fit and generalisation differences, whereas performance on Test Refit represented the final held-out evaluation.

Missing annotation values were replaced with zero. Maximum-absolute scaling was fitted on each LR training fold and applied to its corresponding validation fold, preserving matrix sparsity; RF inputs were not scaled. Confirmed positive genome–environment pairs received sample weight 1.0, whereas unlabelled pairs received weight 0.75. These weights were

combined multiplicatively with inverse class-frequency weights. This procedure reduced the contribution of uncertain negatives but was treated as a heuristic weighting strategy rather than as a formal positive–unlabelled learning algorithm.

Out-of-fold probabilities were accumulated across the three folds. Candidate configurations were ranked first by out-of-fold macro average precision, with out-of-fold macro F1 and micro F1 used as tie-breakers. After hyperparameter selection, each one-vs-rest model was refitted on the complete training set. A separate decision threshold was estimated for each sub-environment from its out-of-fold precision–recall curve by selecting the threshold that maximised F1. The resulting thresholds were applied unchanged to the held-out test set.

Table IV-SM6. **Hyperparameter grids for the sub-environment classifiers.**

Model	Hyperparameter	Values evaluated
Logistic regression	Regularisation strength alpha	1×10^{-5}, 1×10^{-4}, 1×10^{-3}, 0.01, 0.025, 0.05, 0.1
Logistic regression	Elastic-net L1 mixing ratio	0.2, 0.3, 0.5, 0.7, 0.8
Random Forest	Number of trees	100, 200, 300
Random Forest	Maximum tree depth	5, 10
Random Forest	Minimum samples per leaf	5, 7, 8
Random Forest	Features considered per split	$\sqrt{p}$, $\log_2(p)$
Random Forest	Cost-complexity pruning alpha	0.0, 0.2, 0.5, 0.7, 0.9

Fixed LR settings were log_loss, elastic-net penalty, optimal learning-rate scheduling, a maximum of 3,000 iterations, early stopping on an internal 10% validation fraction, 15 non-improving iterations, tolerance 0.001 and random state 420. Fixed RF settings were bootstrap sampling and max_samples = 0.7. The grid comprised 35 LR and 180 RF combinations. For the selected KO-based LR model, the retained parameters were alpha = 0.001 and l1_ratio = 0.2, together with the fixed settings reported above.

Supplementary Figures IV

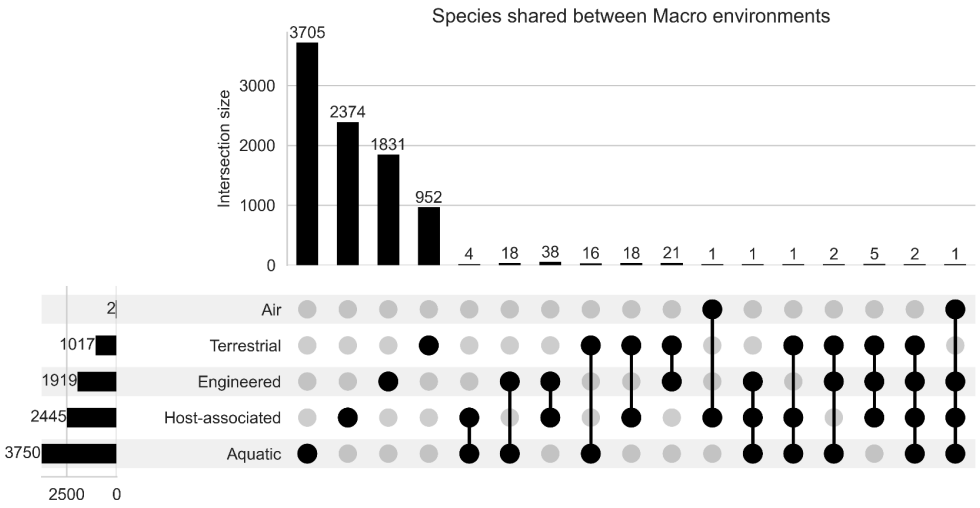


Figure IV-S1. **Distribution and overlap of macro-environmental labels among species-level representative genomes.** Horizontal bars show the total number of representatives associated with each macro-environment, while vertical bars report the number associated with each individual label or combination of labels. Filled circles and connecting lines indicate the macro-environments included in each combination. Genomes associated with Air were excluded from subsequent analyses because of their limited representation. Representatives associated with more than one of the four retained macro-environments were excluded from model development and evaluated separately.

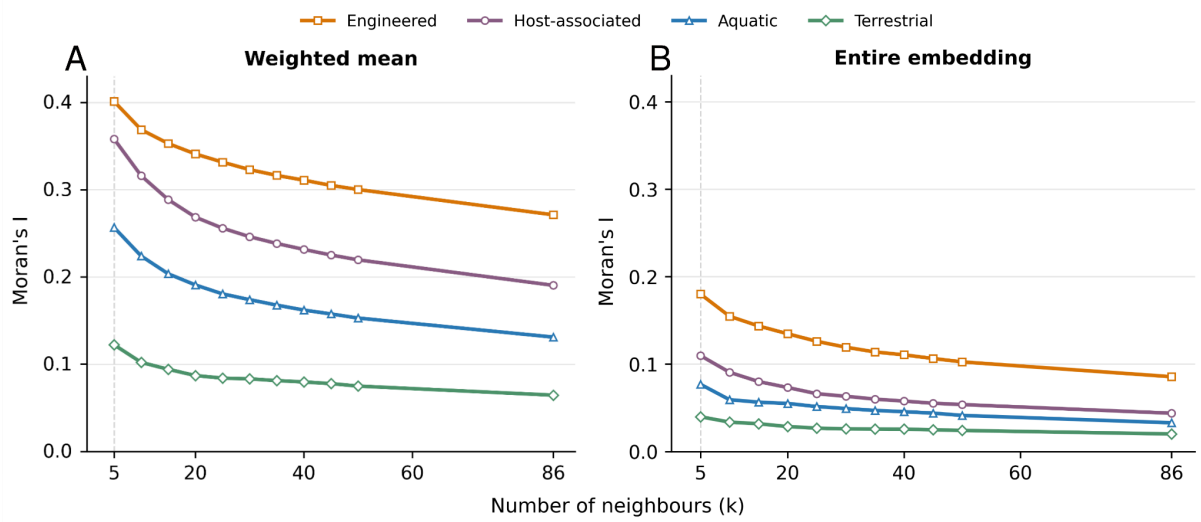


Figure IV-S2. **Moran's I across Evo 2 embedding representations and neighbourhood sizes.** Moran's I was calculated on k-nearest-neighbour graphs constructed from cosine distances among the training-set genome embeddings. Values are shown for the weighted-mean representation (A) and the complete representation obtained by concatenating all four embedding components (B). Lines indicate the four macro-environments and points correspond to the evaluated neighbourhood sizes. The dashed vertical line indicates $k = 5$.

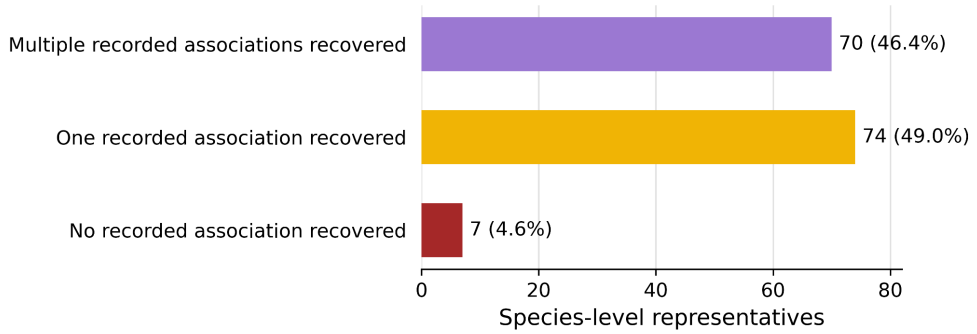


Figure IV-S3. **Exploratory evaluation of multi-environment representatives.** The hierarchical multitask model was applied to 151 species-level representatives associated with more than one macro-environment and excluded from model development. The order and relative values of the probabilities assigned to the four macro-environments were manually compared with the recorded environmental labels. Bars show the number and percentage of representatives for which multiple, one or none of the recorded associations were recovered. As the model was trained for single-label classification and no predefined threshold was used to assign multiple labels, this analysis was considered exploratory.

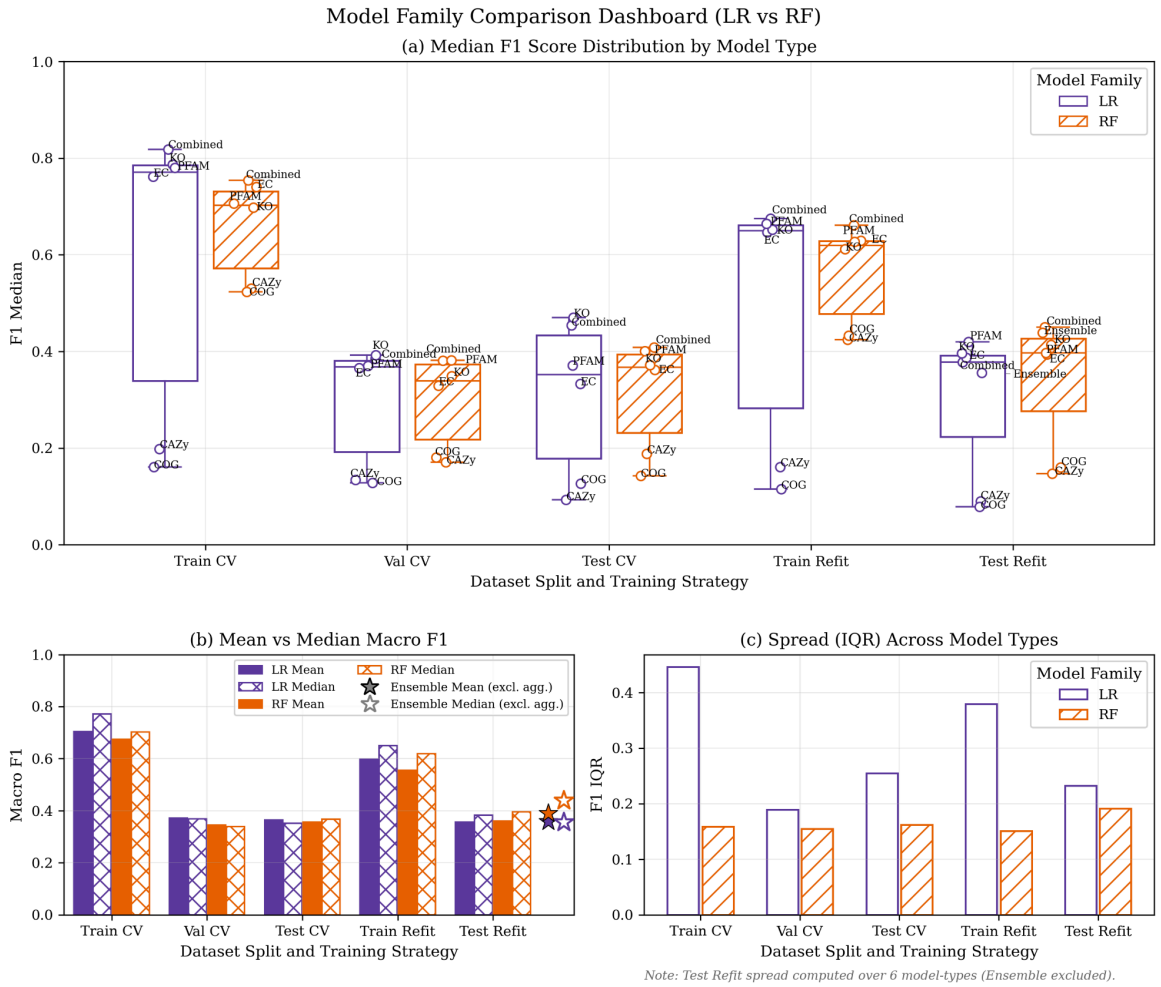


Figure IV-S4. **Comparison of logistic regression and random forest performance across sub-environment model-development stages.** A) Distribution of median class-specific F1 scores for logistic regression (LR) and random forest (RF) models across the five evaluation conditions. Individual points indicate the results obtained using the different functional annotation systems; boxes represent the interquartile range and whiskers the complete range. B) Mean and median macro F1 aggregated by model family and evaluation condition. Stars indicate the mean and median performance of the ensemble on the Test Refit set. C) Interquartile range of median F1 scores across functional annotation systems, indicating the variability observed within each model family.

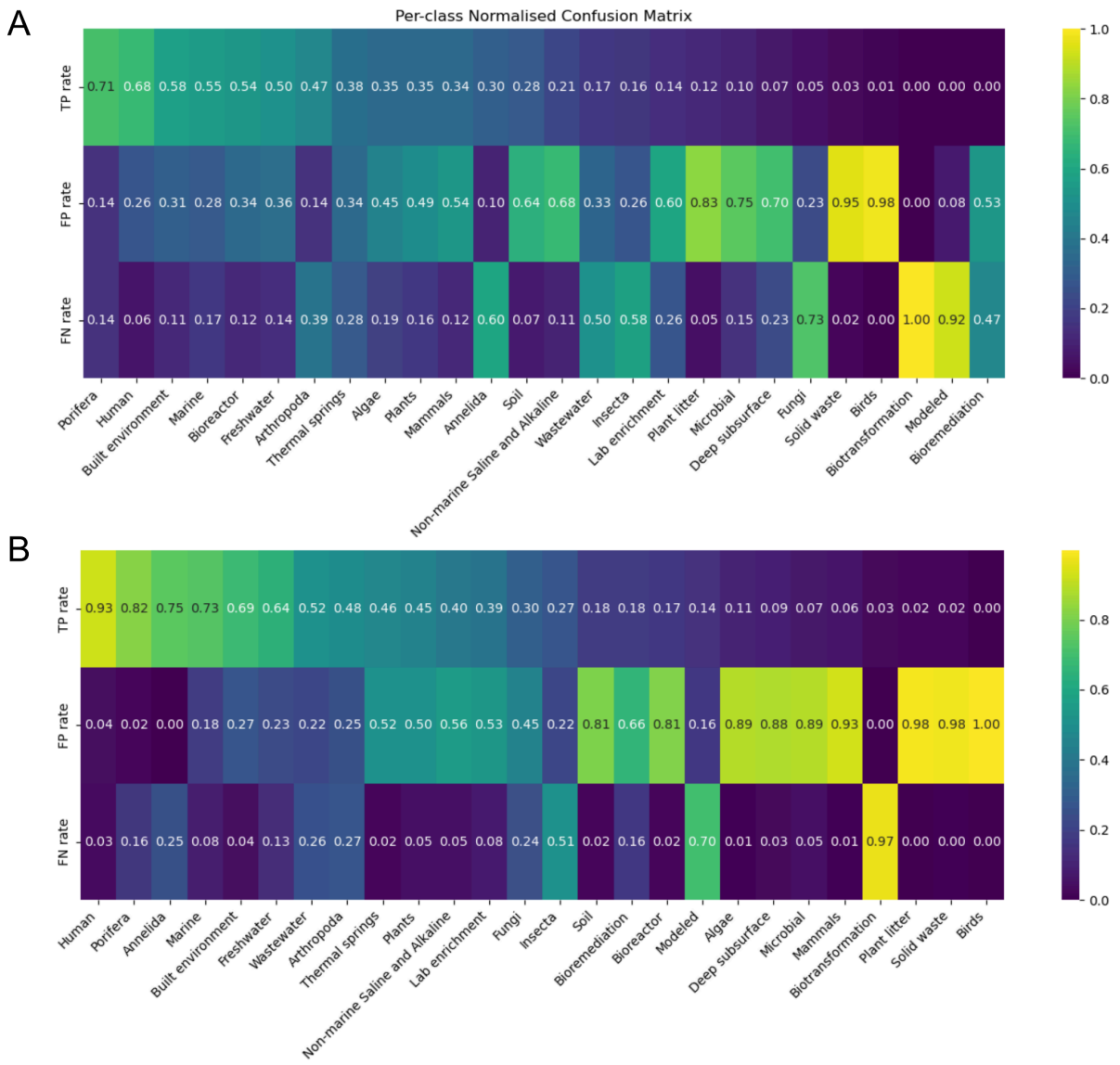


Figure IV-S5. **Class-specific prediction outcomes for the selected KO-based logistic regression model.** True-positive, false-positive and false-negative proportions are shown for the held-out test set (A) and for an additional evaluation dataset containing further genomes from species represented during model development (B). For each sub-environment, TP, FP and FN counts were normalised by their sum, excluding true negatives. Environmental categories are ordered by decreasing TP proportion within each dataset.

Supplementary Tables IV

Table IV-S1. **Genome metadata, quality estimates, environmental annotations and dataset-inclusion information.** The table includes 25,233 MAGs assigned to OTUs as described in the Methods, comprising 22,983 genomes from the GEM catalogue and 2,250 genomes from the AD collection. Completeness and contamination correspond to values recalculated using CheckM2. The RETAINED column identifies the 10,912 species-level representatives retained after clustering. Environmental annotations and taxonomy are reported for these representatives. MACRO_MODEL_STATUS distinguishes the 10,761 representatives included in single-label macro-environment modelling from the 151 representatives excluded because they were associated with multiple macro-environments. Multiple environmental labels associated with the same representative are reported in the same cell and separated by |.